



SISTEMA DE COMERCIO ELECTRÓNICO

Proyecto final - Diseño de Patrones

Tema seleccionado: Opción 7 de la guía: Sistema de Comercio Electrónico

Integrantes	[Completar nombres y apellidos]
Docente	[Completar]
Ciclo / sección	[Completar]
Tecnologías	Java 17, Spring Boot, PostgreSQL, HTML, CSS y JavaScript
Fecha	Julio de 2026

Documento técnico, matriz de rúbrica, pruebas y guía de sustentación

ÍNDICE

1. Introducción
2. Problema, justificación y alcance
3. Objetivos
4. Actores, requisitos y casos de uso
5. Arquitectura del sistema
6. Modelo de dominio y persistencia
7. Patrones GOF implementados
8. Principios SOLID
9. Patrones GRASP
10. Validaciones y seguridad
11. Plan de pruebas y evidencias
12. Matriz de cumplimiento
13. Guía de sustentación
14. Conclusiones y recomendaciones

Resultado esperado: La adaptación integra siete patrones GOF en flujos reales y documenta cada criterio exigido para la categoría Excelente.

1. INTRODUCCIÓN

FastMarket es una plataforma web de comercio electrónico orientada a la publicación, venta y administración de productos. El cliente explora el catálogo, gestiona un carrito, aplica cupones, registra pedidos, selecciona un medio de pago y consulta el historial de compra. Los vendedores administran productos y estadísticas; los administradores controlan usuarios, pedidos, reclamos, contenido, configuración y reportes.

La adaptación se diseñó para responder directamente a la guía del proyecto final. Los patrones no se presentan como ejemplos aislados: Builder participa en la creación del pedido; Factory y State gobiernan su ciclo de vida; Observer registra historial y notifica; Adapter desacopla Mercado Pago; Facade simplifica el checkout; y Proxy protege los reportes.

2. PROBLEMA, JUSTIFICACIÓN Y ALCANCE

2.1 Problema

Una tienda digital coordina usuarios, roles, productos, stock, carrito, descuentos, pedidos, pagos, notificaciones, historial y reportes. Si estas responsabilidades se concentran en clases extensas, el sistema queda fuertemente acoplado, es difícil de probar y cualquier integración nueva obliga a modificar múltiples componentes.

2.2 Justificación

La orientación a objetos, los patrones GOF, SOLID y GRASP permiten asignar responsabilidades de forma estable. La arquitectura por capas separa presentación, controladores, casos de uso, dominio, integraciones y persistencia. Esto mejora mantenibilidad, trazabilidad y extensibilidad.

2.3 Alcance funcional

- Registro, verificación de correo, login y recuperación de contraseña.
- Roles de cliente, vendedor y administrador.
- CRUD de productos, categorías, banners, cupones y contenido.
- Catálogo, búsqueda, filtros, detalle y carrito persistente.
- Checkout con entrega, cupón, costo de envío y pago.
- Pedidos con stock, estados, historial y notificaciones.
- Reclamos, estadísticas, reportes y configuración.
- Integración opcional con Mercado Pago, Gmail y asistente IA.

3. OBJETIVOS

3.1 Objetivo general

Diseñar e implementar un sistema de comercio electrónico modular en Java que resuelva la compra y administración de productos, aplicando patrones GOF, los cinco principios SOLID, patrones GRASP, persistencia relacional y arquitectura por capas.

3.2 Objetivos específicos

1. Implementar operaciones de registro, consulta, actualización y eliminación para las entidades administrables.
2. Aplicar al menos dos patrones creacionales, dos estructurales y dos de comportamiento en procesos reales.
3. Mantener alta cohesión y bajo acoplamiento mediante interfaces, servicios especializados y paquetes modulares.
4. Persistir usuarios, productos, carritos, pedidos, cupones, historial y reclamos en PostgreSQL.
5. Validar roles, stock, cantidades, cupones, estados y datos obligatorios.
6. Documentar arquitectura y relaciones mediante UML.
7. Incorporar pruebas unitarias y un plan de pruebas funcionales.

4. ACTORES, REQUISITOS Y CASOS DE USO

Actor	Responsabilidades
Cliente	Registro, catálogo, carrito, cupones, checkout, pedidos, historial y reclamos.
Vendedor	Gestión de sus productos, pedidos relacionados y estadísticas.
Administrador	Usuarios, pedidos, cupones, reclamos, banners, configuración y reportes.

Actor	Responsabilidades
Pasarela de pago	Recibe una preferencia y devuelve la URL del checkout.
Servicio de correo	Verificación, recuperación y notificaciones de estado.

4.1 Casos de uso principales

Código	Caso de uso	Actor
CU01	Registrar y verificar usuario	Cliente
CU02	Iniciar sesión según rol	Todos
CU03	Buscar y consultar productos	Cliente
CU04	Gestionar productos	Vendedor/Admin
CU05	Administrar carrito	Cliente
CU06	Aplicar cupón	Cliente
CU07	Crear pedido y reservar stock transaccionalmente	Cliente
CU08	Generar preferencia de pago	Cliente
CU09	Cambiar estado del pedido	Admin
CU10	Consultar historial	Según autorización
CU11	Registrar/atender reclamos	Cliente/Admin
CU12	Generar reportes protegidos	Admin

5. ARQUITECTURA DEL SISTEMA

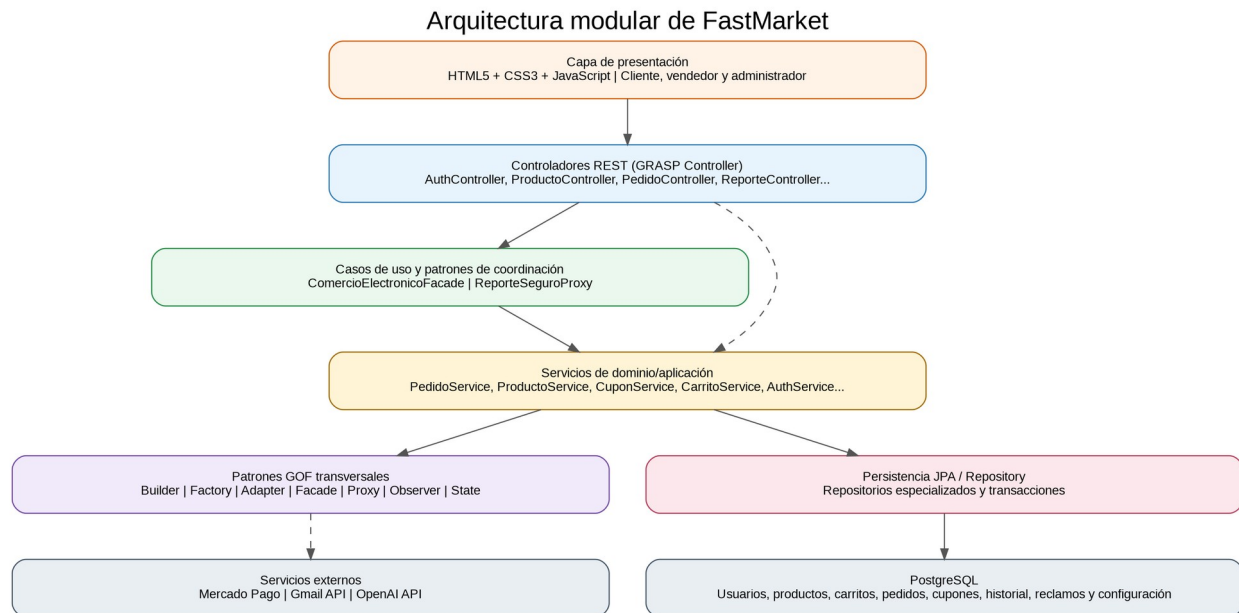


Figura 1. Arquitectura modular y dependencias principales de FastMarket.

La dirección principal es presentación → controlador → fachada/servicio → repositorio o servicio externo. Los controladores reciben solicitudes HTTP y delegan; los servicios contienen reglas del negocio; los repositorios encapsulan datos; y los patrones resuelven variaciones de creación, estructura y comportamiento.

5.1 Organización de paquetes

```

com.fastmarket.api
├── config      configuración y beans
├── controller  endpoints REST
├── dto         contratos de entrada/salida
├── model       entidades y enumeraciones
├── repository  persistencia JPA
├── service     casos de uso
├── pattern
│   ├── creational  builder, factory
│   ├── structural  adapter, facade, proxy
│   └── behavioral  observer, state
  
```

5.2 Flujo principal de compra

1. El cliente selecciona productos y actualiza su carrito.
2. El controlador delega el checkout a ComercioElectronicoFacade.
3. PedidoService valida ítems, producto activo, cantidad y stock.
4. PedidoBuilder construye pedido, ítems y totales.
5. JPA persiste el pedido, descuenta stock y publica el evento para historial y correo.
6. Adapter prepara el pago; Factory y State validan las transiciones posteriores.

6. MODELO DE DOMINIO Y PERSISTENCIA

FastMarket - Diagrama UML resumido de clases y patrones

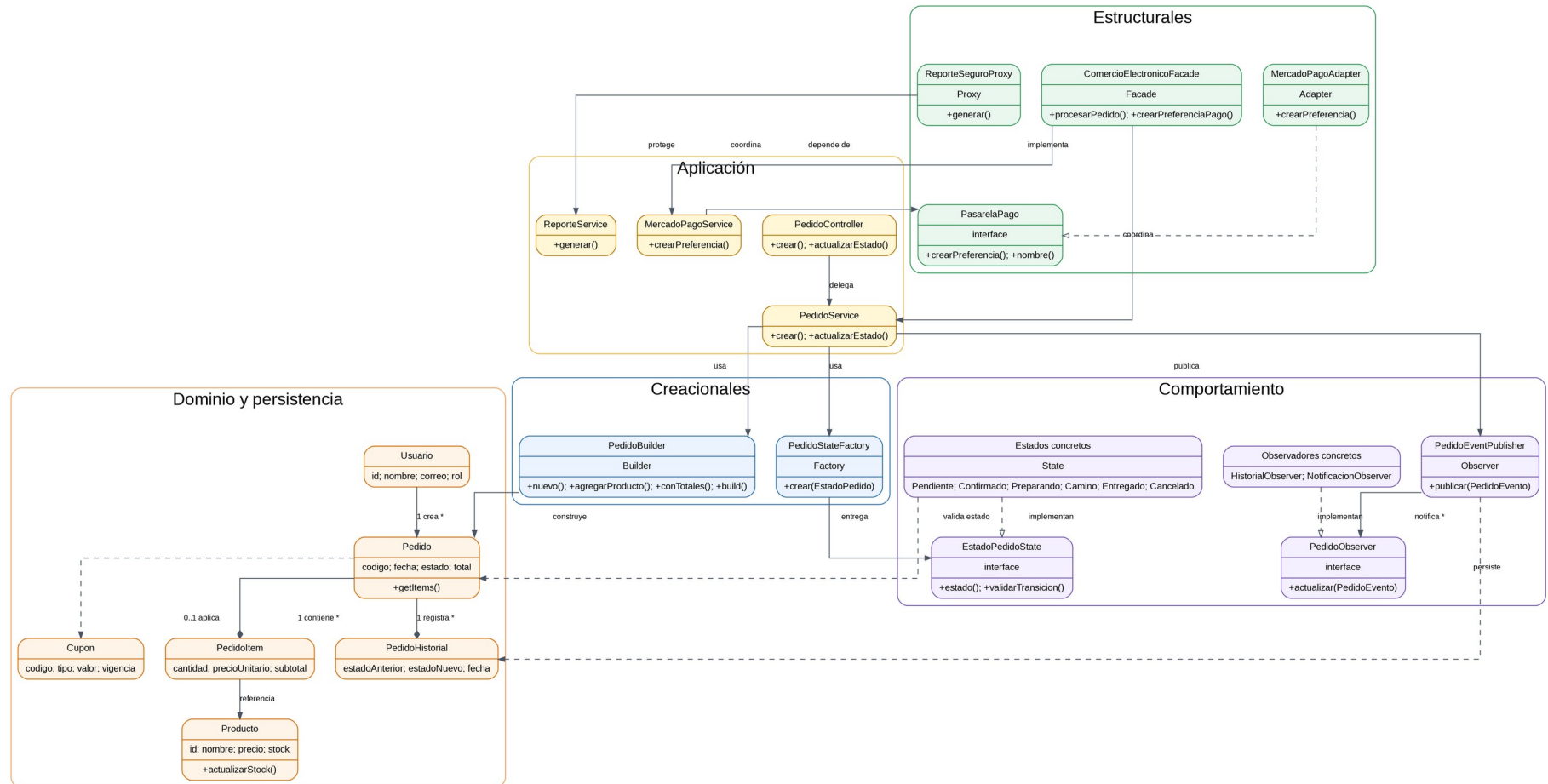


Figura 2. Diagrama UML resumido: dominio, aplicación y siete patrones GOF.

Lectura del diagrama: Naranja: dominio; amarillo: servicios; azul: creación; verde: patrones estructurales; morado: comportamiento.

6.1 Entidades y persistencia

El backend contiene 24 clases de modelo y 15 repositorios. Entre las entidades principales se encuentran Usuario, Direccion, Producto, Categoria, Carrito, CarritoItem, Pedido, PedidoItem, PedidoHistorial, Cupon, CuponUso, Reclamo, Banner, IndexContenido, LoginEvento y SystemConfig.

Relación	Finalidad
Usuario → direcciones/pedidos	Identidad, entrega e historial de compras.
Vendedor → productos	Propiedad y administración del catálogo.
Carrito → ítems	Selección persistente antes del checkout.
Pedido → ítems	Detalle transaccional con precio y vendedor históricos.
Pedido → historial	Trazabilidad de estados y actores.
Cupón → usos/pedidos	Control de descuento, vigencia y límite de uso.

6.2 Nivel de persistencia

- PostgreSQL como motor relacional.
- Spring Data JPA para repositorios y consultas.
- Transacciones en operaciones críticas de pedido y stock.
- Paginación, ordenamiento y consultas por usuario/vendedor.
- Historial persistente de cambios.
- Configuración por variables de entorno y Docker Compose.

7. PATRONES GOF IMPLEMENTADOS

Se integraron siete patrones: dos creacionales, tres estructurales y dos de comportamiento. La guía exige seis; por tanto, la implementación supera el mínimo.

Categoría	Patrón	Aplicación
Creacional	Builder	Construcción consistente del pedido.
Creacional	Factory	Selección del State actual.
Estructural	Adapter	Traducción del contrato de Mercado Pago.
Estructural	Facade	Entrada simple para checkout y pago.
Estructural	Proxy	Autorización antes de reportes.
Comportamiento	Observer	Historial y notificación ante eventos.
Comportamiento	State	Transiciones válidas del pedido.

7.1 Builder - PedidoBuilder

Problema: Crear un pedido exige usuario, código, entrega, pago, cupón, varios ítems y totales. Una secuencia de setters puede dejar un objeto incompleto.

Solución: El Builder expone nuevo, conCodigo, conEntrega, conMetodoPago, conCupon, agregarProducto, conTotales y build.

Integración: PedidoService.crear usa el Builder después de validar disponibilidad y stock.

Beneficio: Construcción legible, validación centralizada y facilidad para extender datos de entrega.

7.2 Factory - PedidoStateFactory

Problema: El servicio no debe conocer todas las clases concretas de estado ni contener un switch creciente.

Solución: La Factory registra States en un EnumMap y retorna el correspondiente al enum persistido.

Integración: PedidoService solicita el State actual antes de cambiar el estado.

Beneficio: Punto único de selección y menor acoplamiento.

7.3 Adapter - MercadoPagoAdapter

Problema: El contrato interno no coincide con payload, cabeceras y endpoint de Mercado Pago.

Solución: PasarelaPago define el contrato propio y MercadoPagoAdapter traduce la solicitud.

Integración: MercadoPagoService depende de la interfaz PasarelaPago.

Beneficio: Proveedor sustituible y pruebas con implementaciones falsas.

7.4 Facade - ComercioElectronicoFacade

Problema: El controlador no debe coordinar cada subsistema del checkout.

Solución: La Facade ofrece procesarPedido y crearPreferenciaPago como operaciones de alto nivel.

Integración: PedidoController y MercadoPagoController delegan en la fachada.

Beneficio: Controladores delgados y coordinación centralizada.

7.5 Proxy - ReporteSeguroProxy

Problema: Los reportes administrativos requieren autorización antes de ejecutar consultas.

Solución: El Proxy valida el rol y luego delega en ReporteService.

Integración: ReporteController utiliza el Proxy.

Beneficio: Seguridad, auditoría o caché pueden agregarse sin modificar el servicio real.

7.6 Observer - eventos de pedido

Problema: Historial y correo son reacciones independientes que no deben acoplarse al servicio de pedido.

Solución: PedidoEventPublisher publica PedidoEvento a una colección de PedidoObserver.

Integración: PedidoHistorialObserver persiste; PedidoNotificacionObserver solicita el correo.

Beneficio: Nuevas reacciones se agregan sin tocar el publicador.

7.7 State - ciclo de vida del pedido

Problema: Las transiciones dependen del estado actual y un condicional central sería difícil de mantener.

Solución: Cada State concentra las transiciones permitidas; entregado y cancelado son terminales.

Integración: La Factory selecciona el State y este valida antes de persistir.

Beneficio: Polimorfismo, reglas localizadas y prevención de inconsistencias.

8. PRINCIPIOS SOLID

Principio	Aplicación concreta
S - Responsabilidad única	Controladores reciben HTTP; servicios aplican reglas; repositorios persisten; Adapter traduce; cada Observer y State tiene una función.
O - Abierto/cerrado	Se agregan pasarelas, observadores o estados mediante implementaciones sin reescribir consumidores.
L - Sustitución de Liskov	Cualquier implementación de PasarelaPago, PedidoObserver o EstadoPedidoState respeta el mismo contrato.
I - Segregación de interfaces	Interfaces pequeñas y orientadas a una capacidad específica.
D - Inversión de dependencias	MercadoPagoService depende de PasarelaPago; Publisher depende de PedidoObserver.

Ejemplo DIP: La prueba MercadoPagoServiceTest inyecta una pasarela falsa y demuestra que el servicio no depende de la infraestructura real.

9. PATRONES GRASP

GRASP	Evidencia
Controller	PedidoController, ProductoController y AuthController reciben eventos y delegan.
Information Expert	Servicios especializados y Builder reúnen las reglas/datos requeridos.
Creator	PedidoBuilder crea Pedido y PedidoItem.
Low Coupling	Interfaces y mediadores reducen dependencias directas.
High Cohesion	Clases y paquetes con responsabilidades enfocadas.
Polymorphism	PasarelaPago, PedidoObserver y EstadoPedidoState.
Pure Fabrication	DtoMapper, Facade, Publisher y Proxy.
Indirection	Adapter, Facade, Proxy y Publisher median componentes.

10. VALIDACIONES Y SEGURIDAD

10.1 Validaciones funcionales

- Usuario, código e ítems obligatorios para el pedido.
- Cantidad mayor que cero y agrupación de productos repetidos.
- Producto existente, activo y con stock suficiente.
- Totales y descuentos no negativos.
- Cupón existente, vigente, aplicable y dentro de sus límites.
- Transición de estado permitida por State.
- Acceso al historial según propietario, vendedor relacionado o administrador.
- Acceso a reportes solo para administrador mediante Proxy.

10.2 Seguridad y configuración

- Autenticación mediante token de aplicación y control de roles.
- Servicio especializado para contraseñas.
- Límite de intentos y eventos de login.
- Manejo global de excepciones HTTP.
- CORS configurable por ambiente.
- Credenciales externas fuera del código fuente.
- Trazabilidad: los pedidos cambian de estado en lugar de eliminarse físicamente.

Corrección de seguridad: El token de Mercado Pago que estaba escrito en la configuración original fue reemplazado por la variable MERCADOPAGO_ACCESS_TOKEN.

11. PLAN DE PRUEBAS Y EVIDENCIAS

11.1 Pruebas unitarias incorporadas

Prueba	Comprobación
PedidoBuilderTest	Construcción, ítems, total y rechazo de pedido vacío.
PedidoStateFactoryTest	Flujo progresivo, rechazo de saltos y estados terminales.
MercadoPagoServiceTest	Total calculado en servidor, validación de monto y reembolso de un segundo pago.
PedidoServiceReversionTest	Cancelación con devolución de stock exactamente una vez.
IndexContenidoServiceTest	Filtro de contenido activo aun cuando no se envía el tipo.
ApiExceptionHandlerTest	Respuestas 401/403 y sanitización de errores internos.
PedidoValidationTest	Validación de entrega, método de pago, teléfono e ítems.

11.2 Casos funcionales prioritarios

ID	Caso	Resultado esperado
FT-01	Registro y verificación	Usuario creado sin duplicados.
FT-02	CRUD de producto	Alta, consulta, actualización y baja.
FT-03	Carrito	Ítems y cantidades persistidos.
FT-04	Compra sin stock	Operación rechazada.
FT-05	Cupón	Descuento válido o rechazo explicado.
FT-06	Crear pedido	PENDIENTE, stock reservado y carrito conservado hasta verificar el pago.
FT-07	Transición válida	Estado e historial actualizados.
FT-08	Transición inválida	Solicitud rechazada.
FT-09	Reporte admin	Respuesta permitida.
FT-10	Reporte cliente	Acceso denegado por Proxy.
FT-11	Persistencia	Datos se mantienen después de reiniciar.

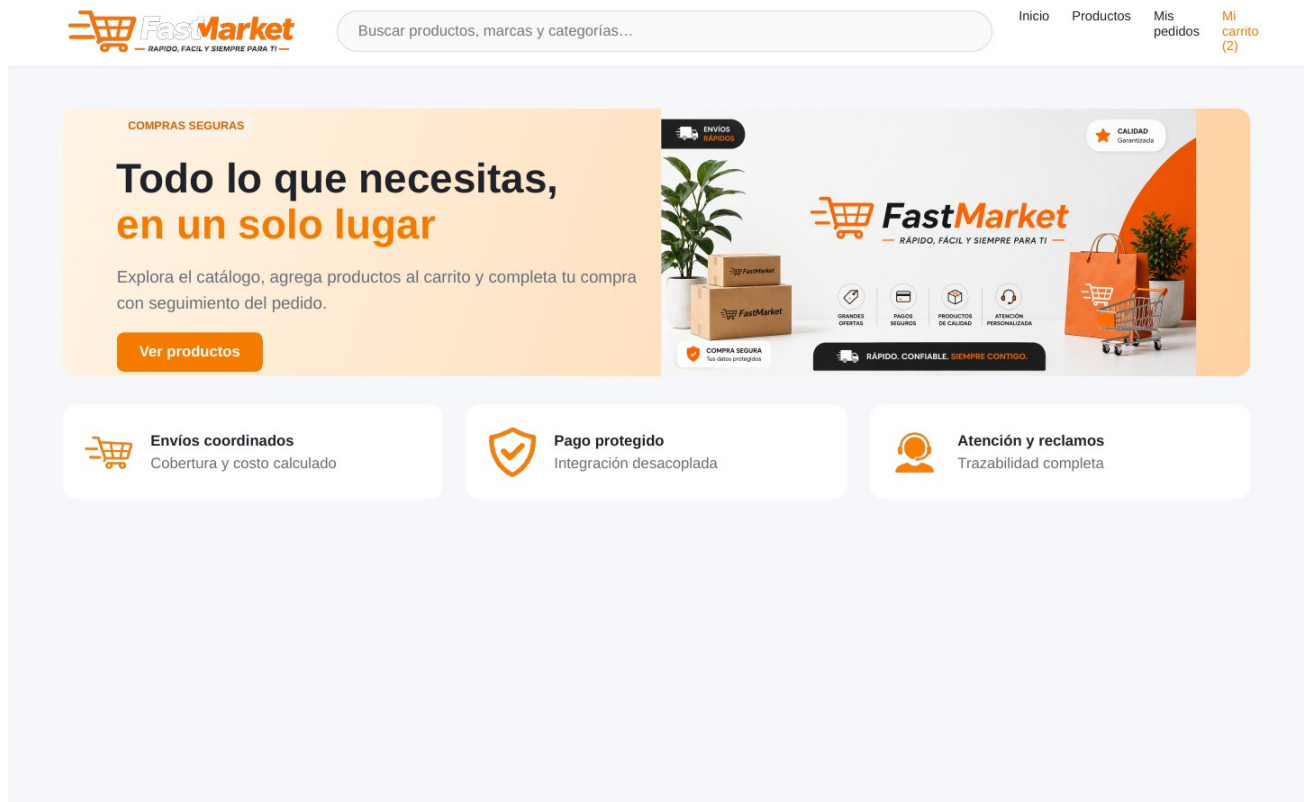


Figura 3. Página de inicio y navegación principal.

La vista principal evidencia el menú de navegación y el acceso a las operaciones centrales del comercio electrónico.

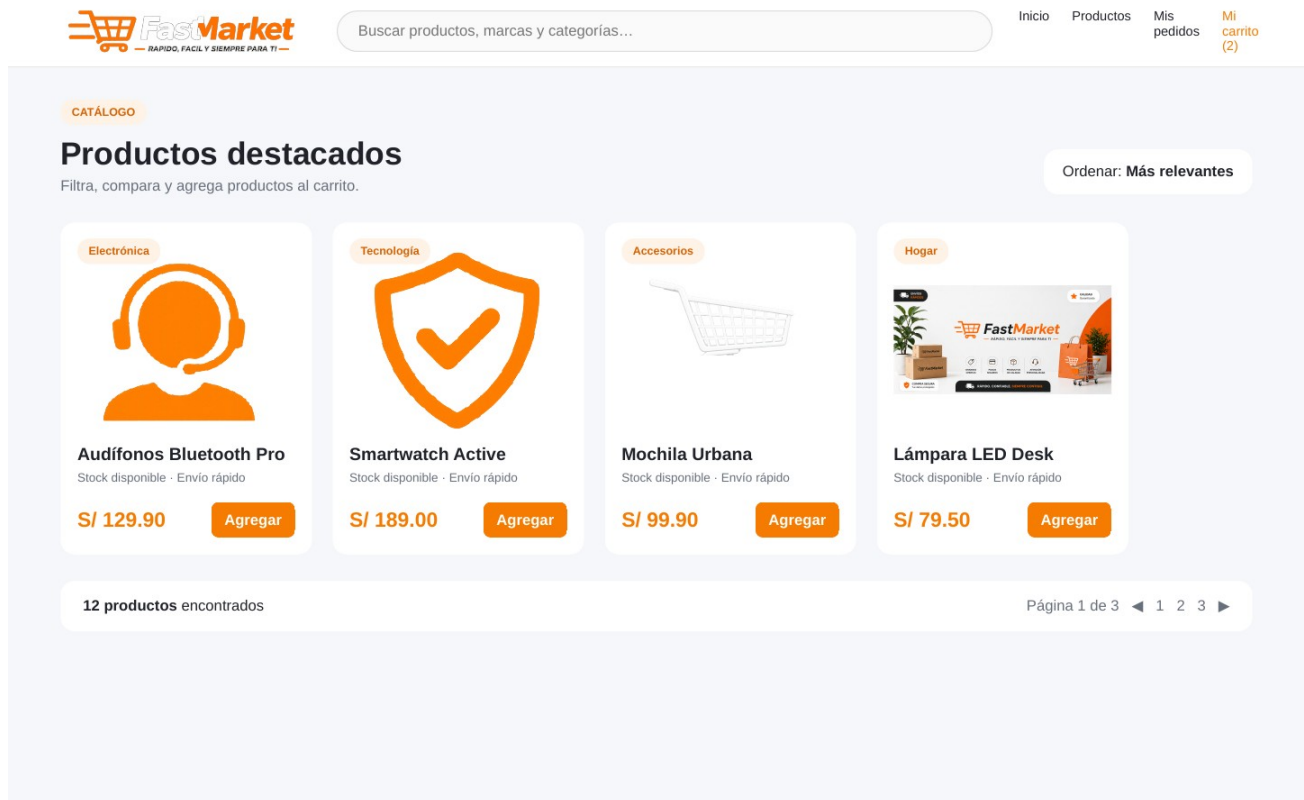


Figura 4. Catálogo, categorías, precios y carrito.

El catálogo presenta colección de productos, clasificación, precio, stock y acción de carrito.



[Inicio](#)
[Productos](#)
[Mis pedidos](#)
[Mi carrito \(2\)](#)

CHECKOUT

Finaliza tu compra

1. Datos de entrega

Nombre

Teléfono

Dirección

2. Método de pago

Mercado Pago

Pago contra entrega

Seleccionado ✓

Resumen del pedido



Audífonos Bluetooth Pro
 Cantidad: 1

S/ 129.90

Subtotal

S/ 129.90

Envío

S/ 8.00

Cupón BIENVENIDA

- S/ 10.00

Total

S/ 127.90

Crear pedido y pagar

El Facade coordina pedido, stock, cupón y pasarela de pago en una sola operación.

Figura 5. Checkout: entrega, pago, cupón y resumen.

El checkout reúne datos de entrega, medio de pago y cálculo de subtotal, descuento, envío y total.

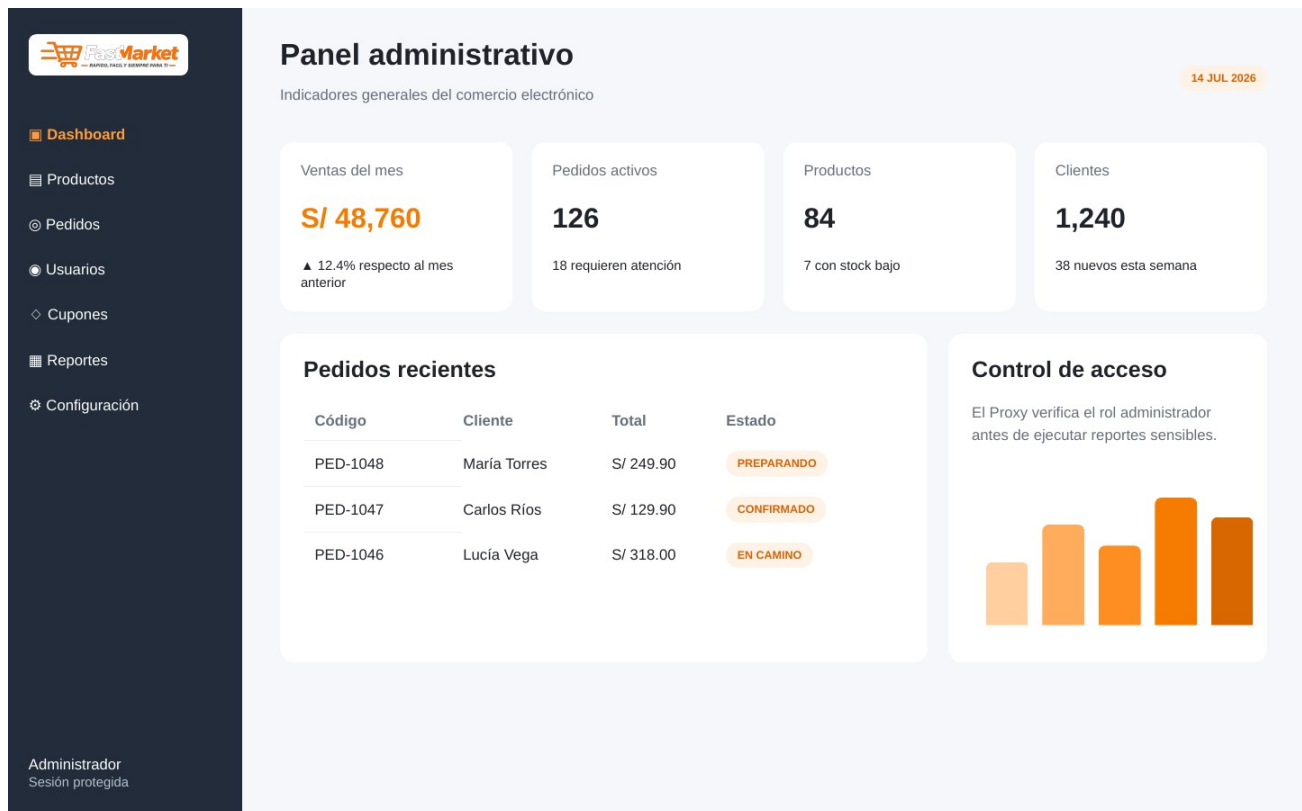


Figura 6. Panel administrativo, indicadores y reporte protegido.

El panel demuestra administración, indicadores y el contexto donde el Proxy controla el acceso a reportes.

12. MATRIZ DE CUMPLIMIENTO

Criterio	Máximo	Evidencia	Proyección
Patrones GOF	6	Siete patrones integrados y justificados.	6
SOLID	3	Los cinco principios con evidencia concreta.	3
GRASP	2	Controller, Expert, Creator, cohesión, acoplamiento, polimorfismo y fabricación pura.	2
Arquitectura y código	3	Paquetes modulares, JPA, configuración y documentación.	3
Funcionalidad	3	Roles, CRUD, carrito, checkout, pedido, pago, historial, reclamos y reportes.	3
Sustentación	2	Guion de 13 minutos, demo y banco de preguntas.	2*
Innovación	1	Pago, correo, historial, cupones, reportes, IA y Docker.	1
TOTAL	20		20*

* La calificación de sustentación depende del dominio oral y de la demostración realizada por el equipo.

12.1 Checklist antes de entregar

- Completar integrantes, docente, curso y sección.
- Copiar .env.example como .env y usar credenciales de prueba.
- Ejecutar ./mvnw clean test y levantar el backend.
- Probar los tres roles y el flujo completo de pedido.
- No incluir archivos .env ni secretos reales.
- Ensayar la exposición con cronómetro.

13. GUÍA DE SUSTENTACIÓN

Tiempo	Contenido
0:00-0:45	Presentación y tema.
0:45-1:45	Problema y objetivo.

Tiempo	Contenido
1:45-3:00	Funcionalidades y actores.
3:00-4:00	Arquitectura.
4:00-8:30	Siete patrones GOF.
8:30-9:45	SOLID y GRASP.
9:45-12:15	Demostración funcional.
12:15-13:00	Conclusiones e innovación.

13.1 Demostración recomendada

1. Abrir catálogo, buscar y agregar producto.
2. Modificar el carrito y aplicar un cupón.
3. Completar checkout y crear pedido.
4. Mostrar la reserva de stock y, después de verificar el pago, la limpieza del carrito.
5. Consultar historial.
6. Cambiar estados válidos como administrador.
7. Intentar una transición inválida.
8. Consultar reporte administrativo y explicar el Proxy.
9. Abrir las carpetas de patrones en el IDE.

13.2 Preguntas técnicas clave

Pregunta	Respuesta central
¿Por qué Builder?	Evita constructor telescópico y pedido incompleto.
¿Por qué State además del enum?	El enum persiste el dato; State encapsula comportamiento y transiciones.
¿Por qué Adapter?	Traduce un proveedor incompatible al contrato interno.
¿Facade y Service son iguales?	La Facade simplifica y coordina varios subsistemas; el Service aplica reglas de un área.
¿Dónde está DIP?	Servicios/publicadores dependen de interfaces, no de implementaciones concretas.
¿Cómo agregar otra pasarela?	Crear otra implementación de PasarelaPago y seleccionarla por configuración/fábrica.
¿Por qué no eliminar pedidos?	Para conservar trazabilidad transaccional e historial.

14. CONCLUSIONES Y RECOMENDACIONES

14.1 Conclusiones

1. FastMarket cumple el tema Sistema de Comercio Electrónico y los flujos centrales de una tienda digital.
2. Los siete patrones GOF participan en procesos ejecutables y superan el mínimo de la guía.
3. SOLID y GRASP se reflejan en contratos pequeños, clases cohesionadas, controladores delegadores y dependencias desacopladas.
4. PostgreSQL y JPA conservan información transaccional, relaciones e historial.
5. State y Observer fortalecen consistencia y trazabilidad; Adapter y Facade preparan el sistema para nuevas integraciones.
6. Proxy separa la política de acceso de la generación real de reportes.

14.2 Recomendaciones

- Ejecutar ./mvnw clean test con Java 17; el proyecto incluye Maven Wrapper.
- Usar credenciales sandbox de Mercado Pago.
- Tener datos de demostración para cliente, vendedor y administrador.
- Mostrar primero el flujo y luego el código de cada patrón.
- Mantener secretos fuera del repositorio.
- Como evolución: Testcontainers, webhooks firmados, auditoría y OpenAPI.

ANEXO A. ESTRUCTURA DE ENTREGABLES

```

ProyectoFinal_ComercioElectronico/
├── backend/                Spring Boot + pruebas
├── frontend/              interfaz HTML/CSS/JS
├── docs/
│   ├── uml/               PNG, DOT y PlantUML
│   ├── evidencias/        capturas
│   ├── API_ENDPOINTS.md
│   └── EVIDENCIAS.md
├── DOCUMENTACION_PROYECTO.md
├── MATRIZ_CUMPLIMIENTO_RUBRICA.md
├── PLAN_PRUEBAS.md
├── GUIA_SUSTENTACION.md
├── VALIDACION_TECNICA.md
├── docker-compose.yml
└── README.md
  
```

ANEXO B. VALIDACIÓN TÉCNICA

Se comprobó sintaxis de los archivos Java con un parser, se revisaron credenciales, se verificaron diagramas y evidencias, y se añadieron siete clases de pruebas unitarias. El entorno de elaboración no contó con Maven operativo y acceso a descarga de dependencias; por ello la compilación completa debe ejecutarse en la computadora de entrega con los comandos siguientes:

```

cd backend
./mvnw clean test
./mvnw spring-boot:run
  
```

IMPORTANTE: No presentar sin ejecutar la compilación y las pruebas en el ambiente final.